FEMU for Next-Generation Storage Research

S0: Setup and architecture

Huaicheng Li

Virginia Tech

October 22, 2026



USENIX ATC '26 tutorial · NSF POSE FEMU Open-Source Ecosystem

https://github.com/MoatLab/FEMU

Why emulate an SSD

The problem

- A new interface is ratified in a standard years before a drive ships
- Real drives are expensive, scarce, and opaque: no firmware, no internals
- A student with no hardware cannot start at all

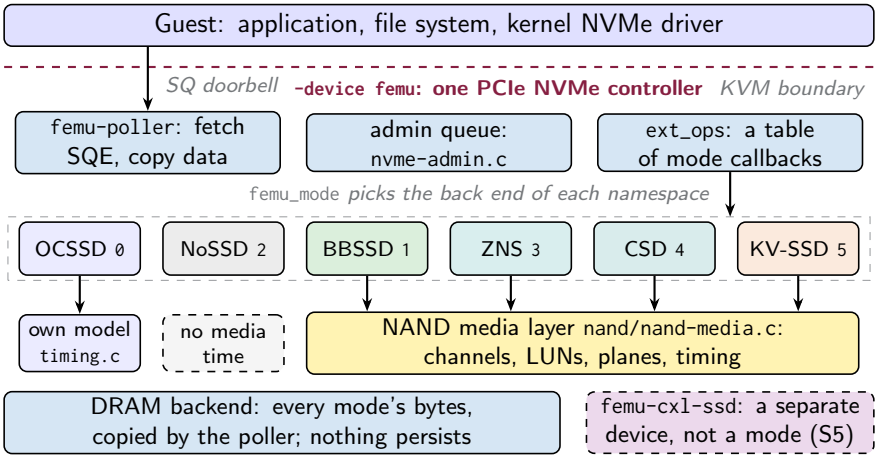
What FEMU gives you

- A full-system NVMe SSD inside a QEMU/KVM guest, on a laptop
- Six device modes, from a plain DRAM disk to computational storage
- The whole FTL, GC and NAND timing in 69 k lines of C you can read

FEMU: *The CASE of FEMU* (FAST '18). As of 2026-10-01: 602 GitHub stars, 264 forks, QEMU 10.1 base, 161 papers in our bibliography build on or cite it.

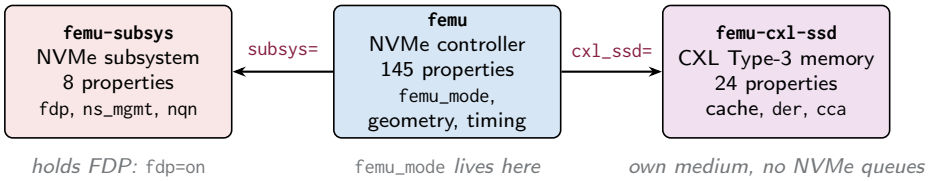
Prototype, test and reproduce a storage design before the hardware exists.

FEMU in one picture



One PCIe device, six mode back ends, one media timing layer, one DRAM image.

Three QEMU devices, not one



- **FDP is not a mode.** It is `fdp=on` on the subsystem, paired with a bbssd controller.
- **CXL-SSD is not a mode either.** It has no NVMe controller; a bbssd `femu` may share its medium.
- 177 static properties across the three. Reference: `hw/femu/docs/properties.md`.

Mode selects the back end of one controller; subsystem and CXL device are separate objects.

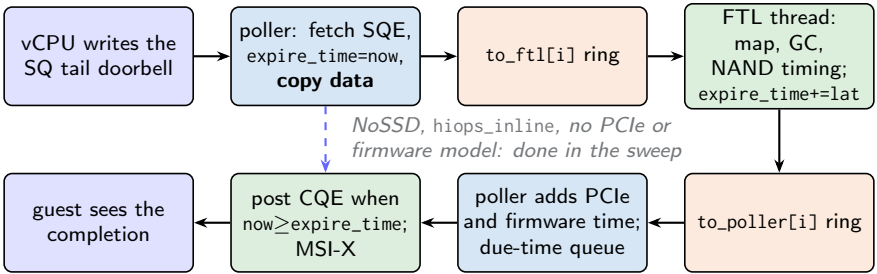
The six modes

femu_mode	Mode	Source	Launcher	Guest kernel
0	OpenChannel 1.2 / 2.0	ocssd/	run-whitebox.sh	4.16 to 5.10 (LightNVM gone in 5.15)
1	BlackBox SSD	bbssd/	run-blackbox.sh	any
2	NoSSD (**default**)	nossd/nop.c	run-nossd.sh	any
3	ZNS	zns/	run-zns.sh	5.10 or newer
4	Computational storage	csd/	run-csd.sh	any; guest tools in tests/csd
5	Key-value SSD	kvssd/	none shipped	any; driven by nvme io-passthru

- femu_mode=6 (SMARTSSD) is a dead enum value: accepted, then every I/O fails
- One controller can mix modes per namespace: namespaces=2,namespace_modes=bbssd,znssd
- Modes 1, 3 and 4 start one FEMU-FTL-Thread per controller; 0 and 5 compute latency in the poller

Six working modes. When in doubt, you are in NoSSD.

A request from doorbell to completion



Without an FTL thread (OCSSD, KV-SSD, NoSSD with a PCIe or firmware model) the poller drains `to_ftl[i]` itself; OCSSD and KV compute their latency in the poller.

Data moves before time passes: the FTL thread decides when, the poller enforces it.

Threads and the knobs that shape them

Thread	How many	Does
QEMU main thread	1	PCI config, MMIO registers, admin queue (synchronous)
`femu-poller`	1, or `ceil(queues/poller_ratio)`	sweep SQs, copy data, post CQEs, raise interrupts
`FEMU-FTL-Thread`	1 per controller	only if a namespace is bbssd, ZNS or CSD
`femu-cxl-ftl`	1 per CXL device	FTL work for the CXL medium
`femu-cxl-cca`	1 if `cca=on`	guest cache-control commands

- `queues=8` by default; `multipoller_enabled=1` gives one poller per queue, `poller_ratio` shards M pollers over N queues
- `hiops_inline` (default on) completes NoSSD I/O inside the sweep when no PCIe link or firmware-CPU model is enabled: no ring, no priority queue
- Rings are `rte_ring` MP/SC (`hw/femu/lib/rte_ring.c`); the due-time queue is `hw/femu/lib/pqueue.c`
- Measured on master (`hw/femu/docs/HIOPS.md`, 2-socket Xeon Gold 6548Y+): SPDK in the guest 80.0M IOPS at 512 B, 37.1M at 4 KiB; kernel io_uring tops out near 3M

Pollers never block; the FTL thread is the one place time is modelled.

A mode is a table of callbacks

hw/femu/nvme.h:

```
typedef struct FemuExtCtrlOps {
    void     *state;
    void     (*init_ctrl_name)(struct FemuCtrl *, struct NvmeNamespace *);
    void     (*init)(struct FemuCtrl *, NvmeNamespace *, Error **);
    void     (*exit)(struct FemuCtrl *);
    void     (*ns_exit)(struct FemuCtrl *, NvmeNamespace *);
    uint16_t (*rw_check_req)(struct FemuCtrl *, NvmeCmd *, NvmeRequest *);
    int      (*start_ctrl)(struct FemuCtrl *);
    uint16_t (*admin_cmd)(struct FemuCtrl *, NvmeCmd *);
    uint16_t (*admin_cmd_cqe)(struct FemuCtrl *, NvmeCmd *, NvmeCqe *);
    uint16_t (*io_cmd)(struct FemuCtrl *, NvmeNamespace *, NvmeCmd *, NvmeRequest *);
    uint16_t (*get_log)(struct FemuCtrl *, NvmeCmd *);
} FemuExtCtrlOps;
```

nvme_register_extensions() in hw/femu/femu.c picks the table from femu_mode; each namespace then takes a copy for its own mode. Reads and writes reach io_cmd after the common opcodes (flush,

Adding a mode means filling this table and one line in the dispatcher (session S6).

The smallest mode: 55 lines

hw/femu/nossd/nop.c, the part that matters:

```
static uint16_t nop_io_cmd(FemuCtrl *n, NvmeNamespace *ns, NvmeCmd *cmd,
                           NvmeRequest *req)
{
    switch (cmd->opcode) {
    case NVME_CMD_READ:
    case NVME_CMD_WRITE:
        return nvme_rw(n, ns, cmd, req);   /* copy; streams bookkeeping elided */
    default:
        return NVME_INVALID_OPCODE | NVME_DNR;
    }
}
int nvme_register_nossd(FemuCtrl *n)
{
    n->ext_ops = (FemuExtCtrlOps) { .init_ctrl_name = bb_init_ctrl_str,
                                    .init = nop_init, .io_cmd = nop_io_cmd };
    return 0;
}
```

NoSSD is the template: the whole mode is a copy and a name string.

Where things live

hw/femu/	lines	what
femu.c	3,039	device realize, properties, namespaces, FTL thread
nvme-admin.c, nvme-io.c	4,046 / 2,043	admin and I/O command handling, pollers
nvme.h	2,800	every structure, the mode enum, the C0h log layout
dma.c, intr.c	161 / 237	PRP and SGL mapping; MSI-X and INTx
nvme-{pi,pel,streams}.c		protection information, persistent event log, streams
bbssd/	12 ftl*.c	FTL: datapath, line GC, four mapping schemes, read cache, FDP, geometry
zns/ ocssd/ csd/ kvssd/		the other mode back ends
nand/nand-media.c		the media layer shared by bbssd and ZNS
cxlssd/	6,082	the CXL device: adapter, cache, CCA
scripts/, scripts/configs/		launchers, ssd-config.sh, seven .conf files
tests/qtest/femu-test.c	18,600	367 qtest cases; also tests/unit, tests/csd
docs/		properties.md, cxlssd.md, HIOPS.md

Everything is under hw/femu/; the README now points there, not to the wiki.

Build

```
# Hosts that can build the QEMU 10.1 base: Ubuntu 22.04 and 24.04 (the CI matrix).
git clone https://github.com/MoatLab/FEMU.git && cd FEMU
mkdir build-femu && cd build-femu
cp ../hw/femu/scripts/femu-copy-scripts.sh .   # ../femu-scripts/ is a symlink to the same place
./femu-copy-scripts.sh .                       # stages femu-compile.sh, pkgdep.sh and the run-*.sh
sudo ./pkgdep.sh                               # build dependencies (Debian/Ubuntu)
./femu-compile.sh                              # configure --enable-kvm --target-list=x86_64-softmmu
./qemu-system-x86_64 -device help | grep femu  # expect: femu, femu-subsys, femu-cxl-ssd
./qemu-system-x86_64 -device femu,help | wc -l # the 145 controller properties
```

- `femu-copy-scripts.sh` stages eight scripts plus `pin.sh` and `ftk/`. `run-cxlssd.sh`, `run-nossd-hiops.sh`, `ssd-config.sh` and `configs/` stay in `hw/femu/scripts/`
- CSD with eBPF programs needs `./femu-compile.sh -enable-csd-ubpf[=path]` (session S4)
- `femu-cxl-ssd` appears only when QEMU is built with CXL devices (it is in the default x86

One script configures and builds; verify with `-device help` before anything else.

Guest image and first launch

```
# Stock run scripts look for ~/images/u20s.qcow2; the tutorial image is Ubuntu 24.04.
cd ~/images && qemu-img create -f qcow2 -b u24s.qcow2 -F qcow2 lab.qcow2  # reset = rm
# Inside the guest, once:
#   serial console  GRUB_CMDLINE_LINUX="console=ttyS0,115200 console=tty"
#   any NIC name    netplan: match: {name: "en*"}, dhcp4: true  (ens3 or enp0s3)
#   tools           sudo apt install nvme-cli fio ndctl daxctl
cd ~/FEMU/build-femu && OSIMGF=~/images/lab.qcow2 femu-lab.sh bbssd
ssh -p 8080 femu@localhost
```

- The default bbssd: 12 GiB exposed on 16 GiB of NAND, and 12 GiB of host RAM for the backing store
- Nothing on the emulated device survives QEMU. Copy results out before you quit

The image name, the serial console and the NIC name break first runs.

Verify, benchmark, read the counters

```
sudo nvme list                          # /dev/nvme0n1, model string names the mode
sudo nvme id-ctrl /dev/nvme0 | grep -E 'vid|mn'   # vid 0x1d1d
sudo fio --name=w --filename=/dev/nvme0n1 --rw=randwrite --bs=4k --direct=1 \
         --size=1G --runtime=20 --time_based --iodepth=32 --ioengine=libaio
# Write amplification and media counters: vendor log page C0h (512 bytes)
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -tu4 -j0 -N4   # WAF x1000
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -tu8 -j8 -N8   # host pages
```

offset	field	offset	field
0	waf_x1000 (u32)	32	max_block_reads
8	host_write_pages	40 / 48	read_reclaims / retention_refreshes
16	gc_write_pages	56 / 64	buffer_reads / buffer_read_hits
24	nand_write_pages	72 / 80	buffer_writes / buffer_write_hits

WAF is (nand + gc) / host, read from log page C0h, never from SMART bytes.

Debugging, honestly

Switches that exist

- Run time: `FEMU_EXP_LOG`, `FEMU_DUMP_LPN`, `FEMU_FDP_DEBUG=1`, `FEMU_KV_SELFTEST`, `FEMU_MBE_INTERLEAVE=<node>`
- Compile time: `FEMU_DEBUG_FTL`, `FEMU_DEBUG_NVME`, `FEMU_DEBUG_ZFTL`
- Vendor admin `0xEF`: flip GC delay, delay emulation, accounting

Switches that do not

- `FEMU_DEBUG=1` is read by nothing
- FEMU defines no QEMU trace events
- `gdb-run.sh` launches stock `-device nvme` on an old path

```
sudo gdb --args ./qemu-system-x86_64 -enable-kvm ... -device femu,femu_mode=1,...
(gdb) break femu_realize        # device creation
(gdb) break nvme_process_sq_io  # every I/O fetch
(gdb) break ssd_write           # bbssd write path, hw/femu/bbssd/ftl-datapath.c
```

Grep the source for the variable before you set it.

Lab S0: build, boot, measure

1. **Step 1.** Confirm the prework build: `-device help` lists three FEMU devices. If you have no build, use the lab machine assigned to you.
2. **Step 2.** Image. Make an overlay of the Ubuntu 24.04 tutorial image. Lab hosts have it; your laptop needs the install.
3. **Step 3.** Boot `femu-lab.sh bbssd`; ssh to port 8080, user `femu`.
4. **Step 4.** Verify `nvme list`, `nvme id-ctrl`: note the model string and `vid`.
5. **Step 5.** Write 1 GiB with the fio line, then read C0h twice, a minute apart. Which fields moved?
6. **Step 6.** Break it. Launch with `femu_mode=6` on purpose. What does the guest see? Why?

If the build fails: `-disable-werror` in `femu-compile.sh`, `libslirp-dev` missing, or a host older than 22.04.

Fifteen minutes. Nobody leaves this room without a running FEMU, theirs or ours.

Where we go from here

	Session	You will
1:45	S1 NoSSD and BlackBox	measure write amplification and change one knob
2:30	S2 FDP	place data by hint, and measure when that lowers WAF and when it cannot
3:00	S3 ZNS	manage zones with `nvme zns` and F2FS
3:30	S4 Computational storage	run a program on the device
4:00	S5 CXL-SSD	launch the CXL device and read its cache counters
4:30	S6 Extending and contributing	add a command, add a test, open a PR

Appendices in the handout: NAND and NVMe background, performance and fidelity, research built on FEMU, KV-SSD and OpenChannel, the NVMe feature set (namespaces, PI, PEL, streams), the full lab book.

Every session ends at a keyboard.

FEMU for Next-Generation Storage Research

S1: The BlackBox SSD

Huaicheng Li

Virginia Tech

October 22, 2026



USENIX ATC '26 tutorial · NSF POSE FEMU Open-Source Ecosystem

https://github.com/MoatLab/FEMU

Start from nothing: NoSSD as the baseline

NoSSD (`femu_mode=2`, the default)

- A DRAM disk behind a real NVMe controller
- No FTL, no NAND timing: what you measure is the host, KVM and the NVMe path
- Run it first, so you know what the BlackBox numbers are made of

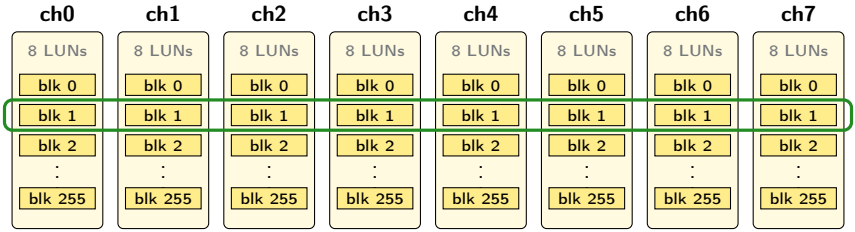
BlackBox (`femu_mode=1`)

- An FTL inside the device: mapping, garbage collection, wear
- NAND latency charged per page, per LUN
- The guest sees an ordinary NVMe SSD

4 KiB, historical Oct. 1 run	**NoSSD**	**BlackBox**	what the model adds
random read, QD1, mean	6.7 µs	51 µs	`pg_rd_lat` 40 µs
random write, QD32	325k IOPS	141k IOPS	`pg_wr_lat` 200 µs
mean latency	98 µs	226 µs	$32/226\,\mu s = 141k$

Measure the host baseline; modeled delays and host work can overlap.

From geometry to lines



A line = block k of all 64 LUNs = 64×256 pages $\times$ 4 KiB = 64 MiB

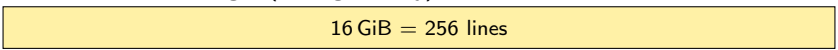
256 blocks per plane give **256 lines** = **16 GiB of NAND**

The write pointer moves channel first, then LUN, plane, page

GC frees space one line at a time: every reclaim touches all 64 LUNs.

Three capacities, and only one is on the command line

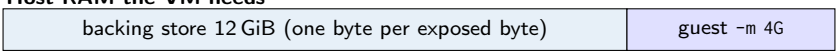
NAND the FTL manages (from geometry)



What the guest sees: `devsz_mb=12288`



Host RAM the VM needs



- `run-blackbox.sh` calls this "25 % over-provisioning": spare over *raw*. In `op_pcent` terms (spare over *exposed*) it is 33 %
- `op_pcent=N` sizes the namespace as NAND$/(1 + N/100)$ for you, and then backs all of the NAND
- Change the geometry and `devsz_mb` is not recomputed. FEMU refuses a namespace larger than the lines left after GC's reserve

Write down all three numbers before you trust an experiment.

A write, inside the FTL

```c
/* bbssd/ftl-datapath.c:611-623, in ssd_write() */
while (should_gc_high(ssd)) {
    /* perform GC here until !should_gc(ssd) */
    r = do_gc(ssd, true);
    if (r == -1)
        break;
}

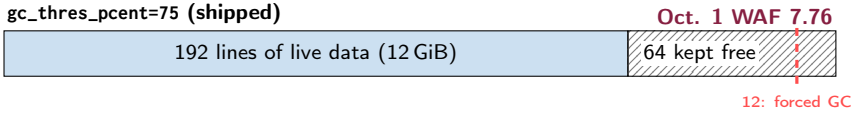
/* refresh at most one read-stressed line per write */
do_read_reclaim(ssd);

/* pages the host wrote, whether or not the buffer absorbs
↪  them */
ssd->host_write_pages += end_lpn - start_lpn + 1;
```

1. **Foreground GC** first, only below the high watermark: the write waits
2. Then each page is programmed **out of place**: new page from the write pointer, old one marked invalid, `nand_write_pages++`
3. The LUN's busy time is the latency: `ssd_advance_status`
4. After the request, **one background GC pass** if below the low watermark

The host pays for GC in two ways: waiting on foreground GC, and queuing behind GC's LUN time.

Why the shipped threshold dominates write amplification



Live data fills every line GC does not keep free: victims are nearly all valid



One knob changed: 30.0k → 65.0k IOPS on the random overwrite

On the default geometry the watermark, not the GC policy, sets write amplification.

The counters: log page C0h

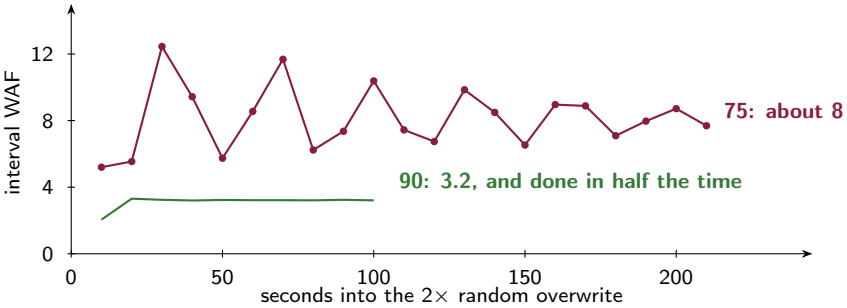
offset	field
0	`waf_x1000` (u32)
8	`host_write_pages`
16	`gc_write_pages`
24	`nand_write_pages`
32	`max_block_reads`
40, 48	read reclaims, retention refreshes
56–80	write-buffer reads, hits, writes

```
nvme get-log /dev/nvme0 -log-id=0xc0
  -log-len=512 -b | od -An -tu8 -w8 -v
```

- WAF $= \dfrac{\Delta(\text{nand} + \text{gc} + \text{map} + \text{pad})}{\Delta\text{host}}$.
  `nand_write_pages` counts *user* pages only, despite its comment
- Counters are cumulative from device start and survive a guest reboot (measured). Always take **interval deltas**
- `waf_x1000` is cumulative too: it hides the steady state behind the fill
- SMART has no NAND write counter, so it cannot give write amplification

Two snapshots and a subtraction. One snapshot tells you about the past.

Historical Oct. 1 example: one controlled experiment



1. Fresh VM. Sequential fill of the whole namespace: on a fresh device this does *no* GC
2. Random 4 KiB overwrite of 2× capacity at QD32; snapshot C0h every 10 s
3. Change **one** knob, fresh VM, repeat. Compare steady-state intervals, not totals

Precondition, sample intervals, change one thing, restart between runs.

Knobs you can turn, one at a time

property	values	what it changes
`gc_thres_pcent, _high`	75, 95	background and forced GC watermarks
`gc_policy`	greedy, random, cost-benefit, fifo, d-choice	victim selection
`mapping`	page, dftl, hybrid, fast	L2P scheme; dftl adds a mapping cache
`cache_evict`	clock, random, lru, arc	mapping-cache eviction (dftl)
`buffer_size`	pages, 0 = off	DRAM write buffer; WAF can fall below 1
`nand_cell_type`	1–4 (SLC–QLC)	per-page-type latency tables
`op_pcent`	percent	namespace size from NAND
`pg_rd/wr_lat, blk_er_lat`	ns	flat NAND timing

Write them as a file: `hw/femu/scripts/ssd-config.sh conf -check` expands an INI file into `-device femu,...` and validates every key against the binary. Seven examples in `scripts/configs/`.

A config file in your repository is a reproducible experiment; a shell history is not.

What survives what

event	data and counters	source
guest reboot	survive	measured on xps
controller reset (`nvme reset`)	survive: FEMU has no device reset handler	code
`simulate-power-loss` (QOM)	buffered writes since the last flush are lost	code
QEMU exits	everything is gone: the backend is DRAM	code

- Power loss needs `power_loss=on`, `buffer_size>0` and `vwc=1`, or nothing is ever lost
- Trigger it with `qom-set /machine/peripheral/nvme0 simulate-power-loss true`; the guest must reset the controller
- Restart QEMU between experiment runs: mapping and line state carry over otherwise

FEMU is a DRAM device that remembers until QEMU exits, unless you ask it to forget.

When the numbers look wrong

symptom	cause
reads faster than `pg_rd_lat`	never-written LBAs: unmapped reads return in under 1 µs. Precondition first
`buffer_reads` non-zero at boot	the guest kernel probed partitions; the counter counts even with no buffer
WAF exactly 1.000	you divided nand by host, or GC never ran (fresh device, only a fill)
latency jitter, warning at start	`cannot pin the ... memory` backend: raise `ulimit -l`
no ssh, console shows a login	guest netplan names another NIC; match en*
QEMU refuses to start	namespace larger than NAND minus GC reserve

Every row here happened while building this tutorial.

Lab S1: occupancy, verified writes, interval counters

1. Boot a fresh BBSSD VM: 12 GiB exposed, 16 GiB raw NAND.
2. Source and export the run's identity fields before preflight.
3. Run `lab-s1-waf.sh` on the explicit disposable namespace.
4. Check 16 GiB of measured writes and successful CRC verification.
5. Compute interval WAF, including mapping and padding writes.
6. Explain GC activity before comparing placement in S2.

Correct bytes and counter accounting are the pass criteria.

What to take to the next session

- The guest sees `devsz_mb`; the FTL sees the geometry; the host pays for both
- WAF is $(\Delta(\text{nand} + \text{gc} + \text{map} + \text{pad}))/\Delta\text{host}$, from C0h deltas
- On the default geometry the GC watermark leaves no slack; change it before comparing policies
- S2 reruns this experiment with **placement hints**: FDP separates hot from cold. Whether that helps depends on the slack you just measured

Next: S2, the same drive with Flexible Data Placement.

FEMU for Next-Generation Storage Research

S2: Placement with FDP

Huaicheng Li

Virginia Tech

October 22, 2026



USENIX ATC '26 tutorial · NSF POSE FEMU Open-Source Ecosystem

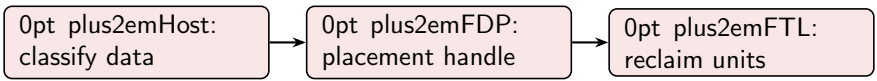
https://github.com/MoatLab/FEMU

Can lifetime hints reduce relocation?

Predict whether hot and cold writes sharing a reclaim unit increase copied pages.

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: `1918eb9795ba`.

Follow the ownership boundary



- The host supplies placement identifiers with writes.
- FEMU resolves the handle inside its endurance group.
- The device still owns mapping and garbage collection.

Keep the comparison controlled

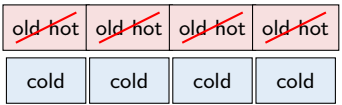
- Same 12 GiB namespace and 16 GiB raw geometry.
- Same fill, seeded request sequence and 16 GiB overwrite volume.
- Fresh VM for each arm; change only placement hints.

Read the right counters

- Log 22h: HBMW, MBMW and MBE are 128-bit byte counters.
- Interval WAF is the change in MBMW divided by the change in HBMW.
- Log 21h reports handle usage, not per-handle byte totals.

Why data lifetimes matter at reclaim time

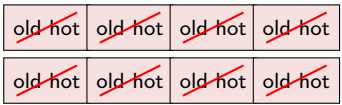
Mixed lifetime unit



Hot data has been overwritten.

Copy live cold pages before reuse.

Separated lifetime unit



All pages in this unit are invalid.

No live-page copy for this unit.

Placement helps when it changes which pages remain live together.

Run the lab and keep the receipt

Host terminal unless labelled Guest.

```
bash labs/bin/preflight.sh labs/guest/lab-s2-fdp.sh /dev/nvme0n1 0
# Restart with a fresh overlay, same configuration.
bash labs/bin/preflight.sh labs/guest/lab-s2-fdp.sh /dev/nvme0n1 1
```

Source connection.env and export its identity fields first. The namespace must be disposable and unmounted.

Check the invariant

1. All fio jobs finish with CRC verification enabled.
2. Both arms write exactly 16 GiB after the same full fill.
3. Compare interval counters; a lower WAF is not a pass requirement.

Diagnose before rerunning

- Missing generic NVMe device: check the guest kernel and nvme driver.
- Unknown FDP option: verify the pinned fio build supports io_uring_cmd.
- No GC activity: revisit occupancy before interpreting placement.

Explain and transfer

When would lifetime separation provide no benefit?

Source pointers

`hw/femu/bbssd/ftl-fdp.c; hw/femu/docs/modes/fdp.md`

FEMU for Next-Generation Storage Research

S3: Zoned storage

Huaicheng Li

Virginia Tech

October 22, 2026



USENIX ATC '26 tutorial · NSF POSE FEMU Open-Source Ecosystem

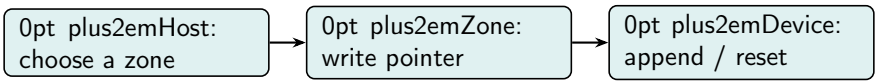
https://github.com/MoatLab/FEMU

What changes when the host owns zone reuse?

After one append, predict the state and write pointer. Does finish erase data?

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: `1918eb9795ba`.

Follow the ownership boundary



- Writes must respect zone boundaries and the current write pointer.
- Append returns the location chosen within a zone.
- Reset makes a zone reusable; finish only closes its writable space.

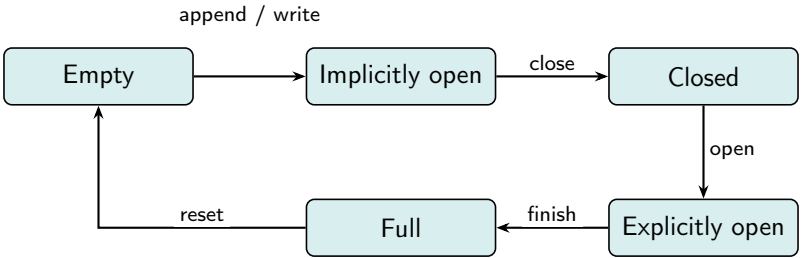
Follow a zone through its states

- Empty: no allocated writes.
- A write opens it; close and open change resource usage.
- Finish makes it full; reset returns it to empty.

Separate three units

- Linux queue/chunk_sectors counts 512-byte sectors.
- NVMe commands address logical blocks, whose size must be read.
- Zone size, writable capacity and transfer size are different.

Check the zone state after each operation



0pt plus2emFinish closes writable space. Reset permits reuse and discards old data.

Run the lab and keep the receipt

Host terminal unless labelled Guest.

```
bash labs/bin/preflight.sh labs/guest/lab-s3-zns.sh /dev/nvme0n1
```

Source connection.env and export its identity fields first. The namespace must be disposable and unmounted.

Check the invariant

1. Readback matches the bytes appended before reset.
2. State and write pointer follow the specified sequence.
3. The invalid write fails with the expected status; reset restores empty.

Diagnose before rerunning

- No namespace: verify CONFIG_BLK_DEV_ZONED and controller identity.
- Unexpected state: preserve the receipt, then reset only the test zone.
- A generic I/O error alone does not identify a write-pointer violation.

Explain and transfer

Which responsibilities move into a filesystem that writes zones?

Source pointers

`hw/femu/zns/zns.c; hw/femu/docs/modes/zns.md`

FEMU for Next-Generation Storage Research

S4: Computational storage

Huaicheng Li

Virginia Tech

October 22, 2026



USENIX ATC '26 tutorial · NSF POSE FEMU Open-Source Ecosystem

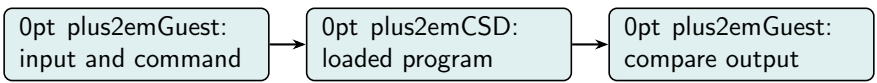
https://github.com/MoatLab/FEMU

Can we verify an offloaded computation?

What would a successful command tell us if the returned data were wrong?

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: `1918eb9795ba`.

Follow the ownership boundary



- Allocate device memory and transfer a deterministic input.
- Load a trusted host program and execute it through the CSD interface.
- Retrieve the result and compare it with the CPU reference.

Two separate checkpoints

- A phantom program checks the command lifecycle.
- A real vector-add library checks computation and returned data.
- Build the guest tool and host library from the same public source pin.

Explain the clock and workers

- nr_cu controls real host workers; different programs can overlap.
- Repeated calls to one loaded program are serialized.
- csd_pause is off in this recipe; wall time includes host execution.

A real program needs a reference result

CPU reference

- Construct deterministic vectors A and B.
- Compute $C_i = A_i + B_i$ independently.
- Retain the expected bytes.

CSD execution

- Copy the same input into device memory.
- Execute the trusted vector-add library.
- Retrieve and compare every output element.

Evidence: matching source revision, tool/library hashes, successful commands, and an exact output comparison.

Command completion and algorithm correctness are separate checks.

Run the lab and keep the receipt

Host terminal unless labelled Guest.

```
# Provision csd-passthru and csd-vadd.so from the pinned source.
bash labs/bin/preflight.sh labs/guest/lab-s4-csd.sh /dev/nvme0n1
```

Source connection.env and export its identity fields first. The namespace must be disposable and unmounted.

Check the invariant

1. The real vector-add result equals the reference output.
2. Allocation, load, activation, execution and cleanup all succeed.
3. Program and tool hashes accompany the transcript.

Diagnose before rerunning

- Program missing: inspect the host csd_program_dir.
- Unknown command: rebuild the guest tool from this source revision.
- A model runtime is not an independent hardware speedup measurement.

Explain and transfer

What data movement should a useful offload eliminate?

Source pointers

`hw/femu/csd/`; `hw/femu/tests/csd/`

FEMU for Next-Generation Storage Research

S5: CXL memory access

Huaicheng Li

Virginia Tech

October 22, 2026



USENIX ATC '26 tutorial · NSF POSE FEMU Open-Source Ecosystem

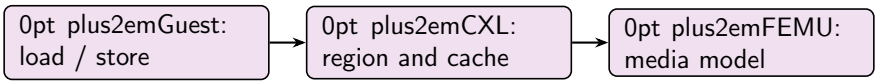
https://github.com/MoatLab/FEMU

How does a memory access reach a storage model?

Which cache must miss before a load reaches the emulated device?

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: 1918eb9795ba.

Follow the ownership boundary



- The guest discovers the CXL topology and creates a RAM region.
- Keep the region in devdax mode for a bounded explicit mapping.
- Compare readback and device counters across a controlled access pattern.

Preflight is part of the experiment

- Use the prepared CXL guest kernel, not an arbitrary distribution image.
- Check CXL, DAX and REGION_INVALIDATION_TEST kernel options.
- Enumerating mem0 is insufficient: create and map a usable region.

Keep the measurement boundary visible

- CPU caches and the FEMU cache are different layers.
- A region must remain outside system RAM for this exercise.
- Use der=off and record the actual cache configuration.

Three checkpoints before interpreting a CXL access

1. **Discovery:** mem0 exists in the guest's CXL topology.
2. **Provisioning:** the decoder commits a region exposed as devdax.
3. **Access:** mapped writes read back correctly and device counters change during the workload.

Why the distinction matters: a kernel may discover the device but refuse region commit because it cannot invalidate the guest's CPU caches.

A listed device is not evidence of a working memory-access path.

Run the lab and keep the receipt

Host terminal unless labelled Guest.

```
# Host: use the CXL-specific image and the recorded QMP socket.
LAB="$PWD/labs" QMP="$RUN_DIR/qmp.sock" bash labs/bin/lab-s5-cxl.sh
```

Source connection.env and export its identity fields first. The namespace must be disposable and unmounted.

Check the invariant

1. A real devdax region is created and mapped.
2. Written bytes read back correctly.
3. The receipt contains before/after cache counters and their scope.

Diagnose before rerunning

- Region creation fails: check guest config and decoder topology.
- Device is system RAM: stop; do not force-convert memory in use.
- No counter change: check CPU caching and whether the mapping was accessed.

Explain and transfer

What would need to change before claiming calibrated load latency?

Source pointers

`hw/femu/cxl/`; `hw/femu/docs/modes/cxl-ssd.md`

FEMU for Next-Generation Storage Research

S6: Extend and reproduce

Huaicheng Li

Virginia Tech

October 22, 2026



USENIX ATC '26 tutorial · NSF POSE FEMU Open-Source Ecosystem

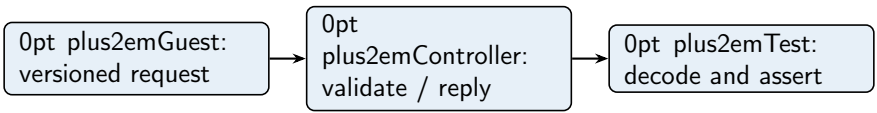
https://github.com/MoatLab/FEMU

What evidence makes a small extension reviewable?

What should fail before the command exists, and what must still fail afterward?

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: `1918eb9795ba`.

Follow the ownership boundary



- Choose an unused teaching opcode at the frozen source revision.
- Return four little-endian fields through the existing PRP helper.
- Reject invalid lengths and reserved fields before transferring data.

A deliberately small interface

- C2h is a teaching opcode, not an upstream reservation.
- Reply: magic, ABI version, mode and byte length.
- The operation is read-only; do not add a timing model.

Red, green, and negative tests

- Unmodified FEMU: the positive request returns Invalid Opcode.
- Patched FEMU: all four fields match the expected values.
- Zero/oversized lengths and nonzero reserved words return Invalid Field.

Run the lab and keep the receipt

Host terminal unless labelled Guest.

```
# Guest, before and after rebuilding the separate checkout:
sudo python3 check-command.py /dev/nvme0 --mode 2
# Host, teaching checkout only:
git apply /path/to/solution.patch
ninja -C build qemu-system-x86_64
```

Source connection.env and export its identity fields first. The namespace must be disposable and unmounted.

Check the invariant

1. Preserve the failing baseline and passing patched receipt.
2. Record both source diffs and binary hashes.
3. Run the regression after restarting the lab VM with the new binary.

Diagnose before rerunning

- Test unexpectedly passes before patch: inspect the actual binary and source.
- Unknown ABI: fail decoding instead of guessing the layout.
- A guest regression is end-to-end evidence, not a qtest result.

Explain and transfer

Can another learner reproduce your claim from the bundle alone?

Source pointers

`labs/extension/README.md; hw/femu/nvme-admin.c`